Amazon Elastic Load Balancing (ELB) — Console Dashboard Wise Interview Questions & Answers

AWS Console Structure

EC2 Console
- Load Balancing
  - Load Balancers
    - Individual Load Balancer
      - Description
      - Network mapping
      - Security
      - Monitoring
      - Listeners and rules
      - Attributes
      - Tags
  - Target Groups
    - Individual Target Group
      - Details
      - Registered targets
      - Health checks
      - Attributes
  - Load Balancer Types
    - Application Load Balancer (ALB)
    - Network Load Balancer (NLB)
    - Gateway Load Balancer (GWLB)

---

1. What is Elastic Load Balancing?

Answer:

Elastic Load Balancing (ELB) is an AWS service that distributes incoming traffic across multiple backend targets.
It helps improve application availability, scalability, and fault tolerance.
ELB can send traffic to EC2 instances, containers, IP addresses, and other supported targets.
AWS provides different load balancer types for different traffic requirements.

For example:

Suppose our website is running on three EC2 instances.
Instead of users directly connecting to one EC2 instance, they connect to the Load Balancer.
The Load Balancer distributes requests across the healthy EC2 instances.

Example diagram:

Users
↓
Load Balancer
↓
┌──────┬──────┐
↓ ↓ ↓
EC2-1 EC2-2 EC2-3

Simple ga gurthupettuko:

ELB = Distribute traffic across healthy targets

---

2. Where do we find Load Balancers in the AWS Console?

Answer:

Load Balancers are managed from the Amazon EC2 Console.
In the left navigation pane, under **Load Balancing**, we can find Load Balancers and Target Groups.
From Load Balancers, we can create, view, modify, and monitor load balancers.
Target Groups are also managed from the same Load Balancing section.

For example:

Suppose we want to create an Application Load Balancer.
We open the EC2 Console and select **Load Balancers** under **Load Balancing**.
From there, we choose the required load balancer type and create it.

Example diagram:

AWS Console
↓
EC2
↓
Load Balancing
- Load Balancers
- Target Groups

Simple ga gurthupettuko:
ELB = EC2 Console → Load Balancing

---

3. What are the main types of Elastic Load Balancers?
Answer:
AWS provides Application Load Balancer, Network Load Balancer, and Gateway Load Balancer.
Application Load Balancer is mainly used for HTTP and HTTPS application traffic.
Network Load Balancer is designed for high-performance TCP, UDP, and TLS traffic.
Gateway Load Balancer is mainly used to deploy and scale network security appliances.
For example:
A web application using HTTP/HTTPS can use an ALB.
A high-performance TCP application can use an NLB.
A security appliance such as a firewall can be placed behind a GWLB.
Example diagram:
ELB
├— ALB → HTTP / HTTPS
├— NLB → TCP / UDP / TLS
└— GWLB → Security Appliances
Simple ga gurthupettuko:
ALB = Application, NLB = Network, GWLB = Gateway

---

4. What is an Application Load Balancer?
Answer:
Application Load Balancer, or ALB, works at the application layer.
It is mainly used for HTTP and HTTPS traffic and can make routing decisions using application-level information.
ALB supports features such as host-based and path-based routing.
It commonly distributes requests to EC2 instances, containers, or IP targets.
For example:
Suppose our application has /users and /orders services.
We can use one ALB and route /users to one target group and /orders to another target group.
This allows different application services to use different backend targets.
Example diagram:
User
↓
ALB
├— /users → Users Target Group
└— /orders → Orders Target Group
Simple ga gurthupettuko:
ALB = HTTP/HTTPS + Smart application routing

---

5. What is a Network Load Balancer?
Answer:
Network Load Balancer, or NLB, operates at the network/transport layer.
It is designed for high-performance traffic such as TCP, UDP, TLS, and supported newer protocols.
NLB can handle very large numbers of connections with low latency.
It is commonly used when application-layer routing from ALB is not required.
For example:
Suppose we have a TCP-based application that requires very high performance.
Instead of using an ALB, we can place an NLB in front of the application servers.
The NLB receives network connections and forwards them to healthy targets.
Example diagram:
Client
↓
NLB
↓
TCP Application
↓
EC2 Servers
Simple ga gurthupettuko:
NLB = High-performance network traffic

---

6. What is a Gateway Load Balancer?
Answer:
Gateway Load Balancer, or GWLB, is designed for deploying, scaling, and managing virtual network appliances.
It is commonly used with firewalls, intrusion detection systems, and other security appliances.
GWLB distributes traffic across multiple appliance instances.
It helps insert network security appliances into the traffic path.
For example:
Suppose a company wants all application traffic to pass through firewall appliances.
GWLB can distribute the traffic across multiple firewall instances.
This allows the security layer to scale without manually managing each firewall.

Example diagram:
Traffic
↓
GWLB
↓
Firewall Appliances
↓
Application
Simple ga gurthupettuko:
GWLB = Load balancing for network appliances

---

7. What is the difference between ALB and NLB?
Answer:
ALB is designed mainly for HTTP/HTTPS application traffic and supports advanced application-level routing.
NLB is designed for high-performance network traffic such as TCP, UDP, and TLS.
ALB can route based on things like host and URL path.
NLB is preferred when very high performance and low latency at the network layer are required.
For example:
If /api and /images need different target groups, ALB is a good choice.
If we need high-performance TCP connections, NLB is generally more suitable.
The choice depends on the protocol and routing requirements.
Example diagram:
ALB → HTTP/HTTPS → Path / Host Routing

NLB → TCP/UDP/TLS → High Performance
Simple ga gurthupettuko:
ALB = Smart HTTP routing | NLB = Fast network routing

---

8. What is a Load Balancer in the AWS Console?
Answer:
The **Load Balancers** page shows the load balancers created in the selected AWS Region.
From this page, we can create new load balancers and open existing load balancers.
We can view their type, state, DNS name, availability zones, and other information.
Opening a load balancer gives access to detailed configuration and monitoring options.
For example:
Suppose we already created an ALB called my-web-alb.
From the Load Balancers page, we select that ALB to see its configuration, listeners, security groups, and monitoring information.
We can also modify supported settings from its detail page.
Example diagram:
EC2 Console
↓
Load Balancers
↓
my-web-alb
↓
Details
Simple ga gurthupettuko:
Load Balancers page = Manage all load balancers

---

9. What is the Load Balancer Description section?
Answer:
The Description section shows the basic information about a load balancer.
It can include the load balancer name, type, state, DNS name, scheme, IP address type, and related configuration.
This section helps us quickly understand what the load balancer is and how it is configured.
It is one of the first places to check when troubleshooting.
For example:
Suppose an ALB is not reachable by users.
We can first check its state, DNS name, scheme, and basic configuration in the Description section.
This helps us confirm whether the load balancer is configured as expected.
Example diagram:
Load Balancer
↓
Description
├— Name
├— Type
├— State
├— DNS Name
└— Scheme
Simple ga gurthupettuko:
Description = Basic Load Balancer information

---

10. What is the DNS name of a Load Balancer?
Answer:
A Load Balancer receives an AWS-generated DNS name when it is created.
Clients can use this DNS name to connect to the load balancer.
Route 53 can also be used to connect a custom domain name to the load balancer.
The DNS name provides a stable way to access the load balancer instead of using its changing IP addresses directly.
For example:
Suppose AWS gives our ALB a DNS name.
Our users can access that ALB through its DNS name, while Route 53 can map www.myapp.com to the ALB.
The user then reaches the ALB and the ALB forwards traffic to healthy targets.
Example diagram:
www.myapp.com
↓
Route 53
↓
ALB DNS Name
↓
ALB
Simple ga gurthupettuko:
ALB DNS = AWS-provided address for the Load Balancer

---

11. What is the Scheme of a Load Balancer?
Answer:
The load balancer scheme determines whether it is internet-facing or internal.
An **internet-facing** load balancer receives traffic from clients through the internet.
An **internal** load balancer is used for applications that should be reachable only through private networking.
The scheme is selected when creating the load balancer.
For example:
A public website can use an internet-facing ALB.
A private backend service can use an internal ALB.
The internal ALB can receive traffic from another application inside the VPC.
Example diagram:
Internet
↓
Internet-facing ALB
↓
Application

Private VPC
↓
Internal ALB
↓
Backend Service
Simple ga gurthupettuko:
Scheme = Internet-facing or Internal

---

12. What is Network Mapping in a Load Balancer?
Answer:
Network mapping defines the Availability Zones and subnets where the load balancer operates.
When creating a load balancer, we select the VPC and appropriate subnets.
Using multiple Availability Zones helps improve availability and fault tolerance.
The load balancer creates nodes in the selected Availability Zones.
For example:
Suppose our VPC has subnets in AZ-1 and AZ-2.
We select both subnets while creating the ALB.
Now the load balancer can operate across both Availability Zones.
Example diagram:
VPC
↓
Load Balancer
├— AZ-1 → Subnet-A
└— AZ-2 → Subnet-B
Simple ga gurthupettuko:
Network Mapping = VPC + Availability Zones + Subnets

---

13. Why should we use multiple Availability Zones for a Load Balancer?
Answer:
Using multiple Availability Zones improves application availability.
If one Availability Zone has a problem, traffic can continue through another enabled Availability Zone.
The load balancer can distribute traffic across healthy targets in the enabled zones.
This reduces the dependency on a single Availability Zone.

For example:
Suppose our ALB operates in AZ-1 and AZ-2.
If AZ-1 has a problem, users can still reach the application through AZ-2.
This gives the application better fault tolerance.
Example diagram:
Users
↓
ALB
├— AZ-1 → EC2
└— AZ-2 → EC2

AZ-1 Failure
↓
AZ-2 continues
Simple ga gurthupettuko:
Multiple AZs = Better availability

---

14. What is a Security Group attached to a Load Balancer?
Answer:
A security group controls network traffic allowed to and from a supported load balancer.
For an ALB, we normally allow the required client traffic such as HTTP or HTTPS on the listener ports.
The security group can also control outbound traffic from the load balancer.
Backend target security groups should allow the required traffic from the load balancer.
For example:
Suppose our ALB listens on HTTPS port 443.
We can allow inbound HTTPS traffic to the ALB security group.
The EC2 security group can then allow application traffic from the ALB security group.
Example diagram:
Internet
↓
ALB Security Group
↓
ALB
↓
EC2 Security Group
↓
EC2
Simple ga gurthupettuko:
ALB SG = Control traffic to the Load Balancer

---

15. Why should the EC2 Security Group allow traffic from the Load Balancer?
Answer:
The load balancer needs permission to connect to the backend targets.
Therefore, the target security group should allow traffic on the application port from the load balancer.
This is more secure than allowing the application port from the entire internet.
It creates a controlled traffic path from the load balancer to the backend servers.
For example:
Suppose ALB forwards HTTP traffic to EC2 on port 80.
The EC2 security group can allow port 80 from the ALB security group.
Users do not need direct access to port 80 on the EC2 instances.
Example diagram:
User
↓
ALB
↓
ALB SG
↓
EC2 SG
↓
EC2 :80
Simple ga gurthupettuko:
EC2 SG = Allow application traffic from ALB

---

16. What is Monitoring in the Load Balancer console?
Answer:
The Monitoring section shows metrics that help us understand load balancer behavior.
We can use metrics to monitor request count, errors, latency, connections, and target health-related information.
These metrics are provided through Amazon CloudWatch.
Monitoring helps us identify performance and availability problems.
For example:

Suppose users complain that the website is slow.
We can check load balancer metrics such as latency and request count.
If errors or latency are increasing, we can investigate the target instances and application.
Example diagram:
Users
↓
ALB
↓
CloudWatch Metrics
├— Requests
├— Errors
└— Latency
Simple ga gurthupettuko:
Monitoring = Understand Load Balancer performance

---

17. What are Load Balancer metrics?
Answer:
Load Balancer metrics provide information about traffic and performance.
Depending on the load balancer type, metrics can include request count, response time, active connections, processed bytes, and errors.
These metrics are available in CloudWatch.
They help us detect performance problems and unusual traffic.
For example:
Suppose request count suddenly increases during a sale.
We can check CloudWatch metrics to understand whether the load balancer and targets are handling the increased traffic.
We can also create alarms based on important metrics.
Example diagram:
ALB
↓
CloudWatch
├— Request Count
├— Latency
├— Errors
└— Connections
Simple ga gurthupettuko:
Metrics = Traffic + Performance information

---

18. What is the Listeners and Rules section?
Answer:
Listeners and Rules define how the load balancer accepts and routes client traffic.
A listener checks for connection requests on a configured protocol and port.
Listener rules decide where requests should be sent when their conditions match.
This is especially important for Application Load Balancers.
For example:
Suppose our ALB has an HTTPS listener on port 443.
We can create a rule that sends /api requests to an API target group.
Other requests can go to the default target group.
Example diagram:
Client
↓
ALB :443
↓
Listener
↓
Rules
├— /api → API TG
└— Other → Web TG
Simple ga gurthupettuko:
Listener = Receive traffic | Rule = Decide where it goes

---

19. What is a Listener?
Answer:
A listener checks for connection requests from clients using a configured protocol and port.
For example, an ALB can have an HTTP listener on port 80 or an HTTPS listener on port 443.
The listener has a default action and can also have additional rules.
The listener connects incoming traffic to the appropriate routing configuration.
For example:
Suppose users access our website using HTTPS.
We create an HTTPS listener on port 443 on the ALB.
The listener receives the request and forwards it according to its rules.
Example diagram:

User
↓
ALB :443
↓
HTTPS Listener
↓
Target Group
Simple ga gurthupettuko:
Listener = Port + Protocol + Traffic entry point

---

20. What is a Listener Port?
Answer:
A listener port is the network port on which the load balancer accepts client connections.
Common examples are port 80 for HTTP and port 443 for HTTPS.
The port must match the traffic clients are sending to the load balancer.
Different listeners can be configured for different protocols and ports.
For example:
Suppose our website uses HTTPS.
We create an HTTPS listener on port 443.
Users connect to the load balancer using port 443 and the listener processes the request.
Example diagram:
User
↓
ALB
↓
HTTPS :443
↓
Listener
↓
Target Group
Simple ga gurthupettuko:
Listener Port = Where ALB accepts traffic

---

21. What is a Default Listener Rule?
Answer:
Every listener has a default rule that handles requests when no higher-priority rule matches.
The default action commonly forwards traffic to a target group.
It provides a fallback routing behavior for the listener.
Additional rules can be created with conditions and priorities.
For example:
Suppose our ALB has a rule for /api.
If a user requests /home, the /api rule does not match.
The request is then handled by the listener's default rule.
Example diagram:
Request
↓
Listener
↓
Rules
├— /api → API TG
└— Default → Web TG
Simple ga gurthupettuko:
Default Rule = Fallback routing rule

---

22. What is a Listener Rule?
Answer:
A listener rule defines conditions and actions for routing traffic.
A rule can inspect information such as host headers, paths, HTTP headers, query parameters, source IP, and other supported conditions.
When the condition matches, the configured action is performed.
Rules have priorities that determine their evaluation order.
For example:
Suppose we want api.myapp.com to go to an API service.
We create a host-header rule for api.myapp.com.
When that hostname is requested, the rule forwards the request to the API target group.
Example diagram:
Request
↓
Listener
↓
Host = api.myapp.com?

↓
API Target Group
Simple ga gurthupettuko:
Listener Rule = Condition + Action + Priority

---

23. What is Rule Priority?
Answer:
Rule priority determines the order in which listener rules are evaluated.
The load balancer checks applicable rules according to their priority until a matching rule is found.
The default rule is evaluated after the other configured rules.
Correct priorities are important when multiple rules could match the same request.
For example:
Suppose we have one rule for /api and another for /.
We give the more specific /api rule the appropriate priority so it is evaluated before the general rule.
This ensures API requests reach the correct target group.
Example diagram:
Listener
↓
Priority 1 → /api → API TG
↓
Priority 2 → / → Web TG
↓
Default
Simple ga gurthupettuko:
Priority = Rule evaluation order

---

24. What is Host-Based Routing?
Answer:
Host-based routing routes requests based on the hostname in the request.
It allows multiple domains or subdomains to use the same Application Load Balancer.
Each hostname can be sent to a different target group.
This is useful for applications with multiple domains or services.
For example:
Suppose we have api.myapp.com and shop.myapp.com.
We can route API requests to an API target group and shop requests to a web target group.
Both applications can use the same ALB.
Example diagram:
ALB
├── api.myapp.com → API TG
└── shop.myapp.com → Shop TG
Simple ga gurthupettuko:
Host Routing = Route based on domain/hostname

---

25. What is Path-Based Routing?
Answer:
Path-based routing routes requests based on the URL path.
It allows different application paths to be sent to different target groups.
This is useful for microservices and applications with multiple backend services.
It can reduce the need to create separate load balancers for every service.
For example:
Suppose /users belongs to the user service and /orders belongs to the order service.
We create listener rules for these paths.
The ALB sends each request to the correct target group.
Example diagram:
ALB
├── /users → Users TG
├── /orders → Orders TG
└── / → Web TG
Simple ga gurthupettuko:
Path Routing = Route based on URL path

---

26. What are Listener Actions?
Answer:
Listener actions define what the load balancer should do when a rule matches.
A common action is forwarding the request to a target group.
Other supported actions can include redirects and fixed responses for Application Load Balancers.
The action depends on the application's routing requirement.
For example:
Suppose a user opens an old HTTP URL.
The listener can redirect the request from HTTP to HTTPS.
For a normal application request, the listener can forward it to a target group.

Example diagram:
Listener Rule
↓
Action
├— Forward
├— Redirect
└— Fixed Response
Simple ga gurthupettuko:
Action = What ALB does after rule matches

---

27. What is Forward Action?
Answer:
A Forward action sends the request to one or more target groups.
The target group contains the backend targets that should receive the traffic.
With supported configurations, traffic can be distributed across multiple target groups using weights.
This is commonly used for normal application traffic.
For example:
Suppose our ALB receives /api requests.
The listener rule can use a forward action to send those requests to the API target group.
The target group then distributes requests to healthy targets.
Example diagram:
Client
↓
ALB
↓
Listener Rule
↓
Forward
↓
API Target Group
↓
EC2
Simple ga gurthupettuko:
Forward = Send request to Target Group

---

28. What is Redirect Action?
Answer:
A Redirect action tells the load balancer to redirect a client request to another URL.
A common use case is redirecting HTTP traffic to HTTPS.
The browser receives the redirect response and makes a new request to the destination URL.
This helps enforce secure access to websites.
For example:
Suppose a user opens http://myapp.com.
The ALB can redirect the request to https://myapp.com.
The browser then connects using HTTPS.
Example diagram:
HTTP :80
↓
ALB Listener
↓
Redirect
↓
HTTPS :443
Simple ga gurthupettuko:
Redirect = Send client to another URL

---

29. What is a Fixed Response Action?
Answer:
A Fixed Response action allows the load balancer to return a predefined response directly.
The request does not need to be forwarded to a backend target for that response.
It can be useful for simple messages, maintenance responses, or specific routing conditions.
The response can include an HTTP status code and configured content.
For example:
Suppose we temporarily disable a particular application path.
Instead of sending the request to an EC2 server, the ALB can return a predefined response.
This can reduce unnecessary backend traffic.
Example diagram:
Client
↓
ALB Rule
↓

Fixed Response
↓
HTTP Response
Simple ga gurthupettuko:
Fixed Response = ALB itself returns response

---

30. What is HTTPS Listener?
Answer:
An HTTPS listener allows the load balancer to accept secure HTTPS connections from clients.
It uses an SSL/TLS certificate to establish encrypted communication with the client.
The certificate is commonly provided through AWS Certificate Manager.
The listener can then forward the request to the backend target group.
For example:
Suppose users access https://www.myapp.com.
The ALB has an HTTPS listener on port 443 with an appropriate certificate.
The ALB terminates the client TLS connection and forwards traffic according to the configuration.
Example diagram:
User
↓ HTTPS :443
ALB
↓
HTTPS Listener
↓
TLS Certificate
↓
Target Group
Simple ga gurthupettuko:
HTTPS Listener = Secure client connection

---

31. What is SSL/TLS Termination at the Load Balancer?
Answer:
SSL/TLS termination means the load balancer handles the encrypted client connection.
The client establishes HTTPS with the load balancer, which uses the configured certificate.
The load balancer can then forward the request to backend targets using the configured protocol.
This removes much of the certificate-handling responsibility from individual application servers.
For example:
A user connects to the ALB using HTTPS.
The ALB handles the TLS connection using an ACM certificate.
The request is then forwarded to the backend application.
Example diagram:
User
↓ HTTPS
ALB
↓ TLS Termination
HTTP/HTTPS
↓
EC2
Simple ga gurthupettuko:
TLS Termination = ALB handles client HTTPS encryption

---

32. What is a Target Group?
Answer:
A Target Group is a collection of registered backend targets that receive traffic from a load balancer.
Targets can include supported resources such as EC2 instances, IP addresses, or containers depending on the load balancer and target group type.
The target group also defines how health checks are performed.
Listeners or listener rules forward traffic to target groups.
For example:
Suppose we have three EC2 instances running our web application.
We register them in a target group called web-tg.
The ALB forwards requests to healthy instances in that target group.
Example diagram:
ALB
↓
Web Target Group
├— EC2-1
├— EC2-2
└— EC2-3
Simple ga gurthupettuko:
Target Group = Group of backend targets

---

33. Where do we find Target Groups in the AWS Console?
Answer:
Target Groups are available under **EC2 → Load Balancing → Target Groups**.
From this page, we can create, view, modify, and delete target groups.
We can also open an individual target group to view registered targets, health checks, and attributes.
Target Groups are an important part of the ELB console.
For example:
Suppose we create an ALB for our web application.
We can create a target group called web-tg from the Target Groups page.
Then we register our EC2 instances with that target group.
Example diagram:
EC2 Console
↓
Load Balancing
↓
Target Groups
↓
web-tg
Simple ga gurthupettuko:
Target Groups = Backend group management page

---

34. What is Target Type in a Target Group?
Answer:
Target type determines what kind of backend targets can be registered in a target group.
Common target types include instance and IP, with other types supported for specific load balancer use cases.
For instance target type, EC2 instances are registered by instance ID.
For IP target type, traffic can be sent to specific IP addresses.
For example:
Suppose our application runs directly on EC2 instances.
We can create an instance-type target group and register the EC2 instances.
If we need to route traffic directly to specific IP addresses, we can use an IP-type target group.
Example diagram:
Target Group
├— Instance → EC2 Instance ID
└— IP → IP Address
Simple ga gurthupettuko:
Target Type = What backend you register

---

35. What is the Protocol and Port of a Target Group?
Answer:
The target group protocol and port define how the load balancer communicates with the registered targets.
For example, a target group may use HTTP on port 80 or HTTPS on port 443.
The selected protocol and port should match the application service running on the targets.
Health checks are also configured based on the target communication setup.
For example:
Suppose our EC2 web servers run an application on HTTP port 8080.
We can create an HTTP target group using port 8080.
The ALB then forwards requests to healthy targets on that port.
Example diagram:
ALB
↓
Target Group
↓
HTTP :8080
↓
EC2 Application
Simple ga gurthupettuko:
Target Group Port = Backend application port

---

36. What are Registered Targets?
Answer:
Registered targets are the actual backend resources associated with a target group.
They can be EC2 instances, IP addresses, or other supported target types.
The load balancer sends traffic to healthy registered targets.
We can register or deregister targets from the target group console.
For example:
Suppose web-tg contains three EC2 instances.
All three are registered targets of the target group.
If one becomes unhealthy, the load balancer stops sending normal traffic to that target.
Example diagram:
web-tg

↓
Registered Targets
├— EC2-1 → Healthy
├— EC2-2 → Healthy
└— EC2-3 → Unhealthy
Simple ga gurthupettuko:
Registered Targets = Backend servers connected to Target Group

---

37. What is a Health Check in a Target Group?
Answer:
A target group health check checks whether registered targets are able to receive traffic.
The load balancer periodically sends health check requests to the configured target and evaluates the response.
Healthy targets can receive traffic, while unhealthy targets are normally removed from normal routing.
Health checks can be configured with protocol, port, path, intervals, and thresholds.
For example:
Suppose our application has a /health endpoint.
We can configure the target group's HTTP health check to call /health.
If the target responds correctly, the load balancer considers it healthy.
Example diagram:
ALB
↓
Health Check
↓
EC2 /health
↓
Healthy / Unhealthy
Simple ga gurthupettuko:
Health Check = Check whether target is ready for traffic

---

38. What is Health Check Protocol?
Answer:
Health check protocol defines the protocol used by the load balancer to check a target.
Depending on the target group and load balancer type, protocols such as HTTP, HTTPS, TCP, and others may be supported.
The health check protocol should match how the application can be checked.
Using the correct protocol helps accurately determine target health.
For example:
Suppose our web application provides an HTTPS health endpoint.
We can configure the target group to use HTTPS health checks.
The load balancer then checks the target using that protocol.
Example diagram:
Target Group
↓
Health Check Protocol
↓
HTTPS
↓
Application
Simple ga gurthupettuko:
Health Check Protocol = How target health is checked

---

39. What is Health Check Path?
Answer:
Health check path is the URL path that the load balancer requests when checking an HTTP or HTTPS target.
The application should provide a suitable endpoint that returns a successful response when it is healthy.
A common example is /health or /.
The configured path should represent the actual health of the application.
For example:
Suppose our application exposes /health.
We configure the target group health check path as /health.
The ALB periodically requests that path to determine whether the target is healthy.
Example diagram:
ALB
↓
GET /health
↓
EC2
↓
200 OK
↓
Healthy

Simple ga gurthupettuko:
Health Check Path = URL used to test application health

---

40. What is Healthy Threshold?
Answer:
Healthy threshold defines how many consecutive successful health checks are required before a target is considered healthy.
It helps prevent a target from being marked healthy because of only one temporary successful response.
The exact supported range depends on the load balancer and health-check configuration.
This setting helps control how quickly a recovered target returns to service.
For example:
Suppose the healthy threshold is configured to require multiple successful checks.
After an EC2 application recovers, the load balancer performs the checks.
Only after the required successful checks does the target become healthy again.
Example diagram:
Target Recovery
↓
Success
↓
Success
↓
Success
↓
Healthy
Simple ga gurthupettuko:
Healthy Threshold = Successful checks needed to become healthy

---

41. What is Unhealthy Threshold?
Answer:
Unhealthy threshold defines how many consecutive failed health checks are required before a target is considered unhealthy.
It prevents a single temporary failure from immediately removing a target from service.
The load balancer continues checking the target based on the configured health check settings.
Once the required failures occur, the target is considered unhealthy.
For example:
Suppose an EC2 server temporarily fails one health check.
The load balancer does not immediately assume the application is permanently down if more failures are required.
If the target continues failing the required number of checks, it becomes unhealthy.
Example diagram:
Target
↓
Failure
↓
Failure
↓
Required failures reached
↓
Unhealthy
Simple ga gurthupettuko:
Unhealthy Threshold = Failed checks needed to become unhealthy

---

42. What is Health Check Interval?
Answer:
Health check interval determines how frequently the load balancer sends health check requests to a target.
A shorter interval detects failures more quickly but results in more frequent health checks.
A longer interval reduces the frequency of checks but may take longer to detect problems.
The correct value depends on the application's availability requirements.
For example:
Suppose the load balancer checks the application regularly.
If the application suddenly stops responding, the next health check can detect the problem.
The unhealthy threshold and interval together influence how quickly the target is removed from normal routing.
Example diagram:
Health Check
↓
Wait Interval
↓
Health Check
↓
Wait Interval
↓
Health Check
Simple ga gurthupettuko:
Interval = How often health check runs

43. What is the Attributes section of a Load Balancer?
Answer:
The Attributes section contains configurable behavior settings for the load balancer.
Depending on the load balancer type, attributes can control things such as deletion protection, access logging, routing behavior, and other features.
These settings change how the load balancer behaves.
We should review them when tuning or securing a production load balancer.
For example:
Suppose we do not want a production ALB to be accidentally deleted.
We can enable deletion protection if supported for that load balancer.
Other attributes can be configured based on application requirements.
Example diagram:
Load Balancer
↓
Attributes
├— Deletion Protection
├— Access Logging
└— Other Behavior Settings
Simple ga gurthupettuko:
Attributes = Load Balancer behavior settings

---

44. What is Deletion Protection?
Answer:
Deletion protection helps prevent accidental deletion of a load balancer.
When enabled, attempts to delete the load balancer are blocked until deletion protection is disabled.
It is useful for important production load balancers.
It does not prevent configuration changes or application failures.
For example:
Suppose our production ALB is critical for the website.
We enable deletion protection so an accidental delete operation does not immediately remove the ALB.
If we intentionally want to delete it, we first disable the protection.
Example diagram:
Production ALB
↓
Deletion Protection
↓
Accidental Delete
X
Simple ga gurthupettuko:
Deletion Protection = Prevent accidental ALB deletion

---

45. What is Access Logging for a Load Balancer?
Answer:
Access logging records detailed information about requests processed by the load balancer.
The logs can help with troubleshooting, traffic analysis, and security investigation.
For supported load balancers, logs can be stored in an S3 bucket.
We can enable and configure access logging from the load balancer settings.
For example:
Suppose we want to investigate which requests reached our application.
We can enable access logging and store the logs in S3.
Later, we can analyze those logs for troubleshooting or auditing.
Example diagram:
Users
↓
Load Balancer
↓
Access Logs
↓
S3 Bucket
↓
Analysis
Simple ga gurthupettuko:
Access Logging = Record Load Balancer requests

---

46. What is Cross-Zone Load Balancing?
Answer:
Cross-zone load balancing allows traffic to be distributed across targets in different Availability Zones.
It can help distribute traffic more evenly when targets are not equally distributed across zones.
The exact behavior and defaults depend on the load balancer type.
It is an important setting to understand when designing multi-AZ architectures.

For example:
Suppose AZ-1 has one target and AZ-2 has three targets.
Cross-zone behavior can allow traffic to be distributed across targets beyond only the local zone.
This can help improve target utilization.
Example diagram:
ALB
├── AZ-1 → EC2-1
└── AZ-2 → EC2-2, EC2-3, EC2-4

Cross-Zone
↓
Traffic can use healthy targets across zones
Simple ga gurthupettuko:
Cross-Zone = Distribute traffic across AZ targets

---

47. What is Sticky Session in a Load Balancer?
Answer:
Sticky sessions allow a client to continue being routed to the same target for a configured period.
This is useful when an application stores session information locally on a backend server.
For ALB, stickiness can be configured using load-balancer-generated or application-based cookies.
It should be used only when the application's design requires session affinity.
For example:
Suppose a user logs into an application and the session is stored locally on one server.
With sticky sessions, subsequent requests from that user can continue going to the same target.
Without stickiness, requests may be distributed to different targets.
Example diagram:
User
↓
ALB
↓
EC2-1
↓
Same User
↓
ALB
↓
EC2-1
Simple ga gurthupettuko:
Sticky Session = Same client → Same target

---

48. What is Deregistration Delay?
Answer:
Deregistration delay gives an existing target time to finish ongoing connections before it is fully removed from service.
This behavior is often called connection draining.
It helps applications finish existing requests gracefully instead of abruptly terminating them.
The timeout can be configured in target group attributes.
For example:
Suppose EC2-1 needs to be removed from the target group.
The load balancer stops sending new requests to that target but allows existing connections time to finish.
After the configured delay, the target is fully deregistered.
Example diagram:
EC2-1
↓
Deregister
↓
No New Requests
↓
Existing Requests Finish
↓
Removed
Simple ga gurthupettuko:
Deregistration Delay = Give existing requests time to finish

---

49. What is a Target Group Attribute?
Answer:
Target group attributes control how traffic is handled for registered targets.
Depending on the load balancer type, attributes can include routing algorithm, stickiness, deregistration delay, cross-zone behavior, and other settings.
These settings are configured from the target group's Attributes section.
They help customize target selection and connection behavior.
For example:

Suppose our application needs sticky sessions.
We open the target group and configure the required stickiness attribute.
The load balancer then uses the configured behavior when selecting targets.
Example diagram:
Target Group
↓
Attributes
├— Routing
├— Stickiness
├— Deregistration
└— Cross-Zone
Simple ga gurthupettuko:
Target Group Attributes = Backend traffic behavior

---

50. What is the Load Balancing Algorithm?
Answer:
The load balancing algorithm determines how requests are distributed among healthy targets.
For Application Load Balancers, supported options include algorithms such as round robin, least outstanding requests, and weighted random depending on configuration and compatibility.
The appropriate algorithm depends on the application's traffic pattern.
It can be configured in target group attributes.
For example:
Suppose three EC2 instances are healthy.
With round robin, requests are distributed sequentially among the targets.
For applications where targets have different workloads, another supported algorithm may be more appropriate.
Example diagram:
ALB
↓
Target Group
↓
Routing Algorithm
├— EC2-1
├— EC2-2
└— EC2-3
Simple ga gurthupettuko:
Algorithm = How targets are selected

---

51. What is Round Robin Routing?
Answer:
Round robin distributes requests sequentially across available healthy targets.
It provides a simple way to spread traffic across multiple targets.
It works well when targets have similar capacity and requests have relatively similar processing requirements.
It is one of the common routing algorithms used with Application Load Balancer target groups.
For example:
Suppose we have three healthy EC2 instances.
The ALB can send requests in sequence to EC2-1, EC2-2, and EC2-3.
The sequence then continues again from EC2-1.
Example diagram:
Request 1 → EC2-1
Request 2 → EC2-2
Request 3 → EC2-3
Request 4 → EC2-1
Simple ga gurthupettuko:
Round Robin = One by one distribution

---

52. What is Least Outstanding Requests?
Answer:
Least outstanding requests routing considers the number of active outstanding requests on the targets.
It can help route new requests toward targets that currently have fewer outstanding requests.
This can be useful when requests take different amounts of time to complete.
It is a supported Application Load Balancer target group routing option.
For example:
Suppose EC2-1 is processing many long-running requests while EC2-2 has fewer outstanding requests.
The load balancer can prefer the target with fewer outstanding requests.
This can help distribute work more effectively.
Example diagram:
ALB
↓
Check Outstanding Requests
├— EC2-1 → 20
├— EC2-2 → 5 ← Prefer

└── EC2-3 → 12

Simple ga gurthupettuko:
Least Outstanding = Prefer target with fewer active requests

---

53. What is Target Health Status?
Answer:
Target health status tells us whether a registered target is ready to receive traffic.
Common statuses include healthy, unhealthy, initial, draining, and other states depending on the service.
The status is based on registration state and health check results.
We can view target health from the Target Group console.
For example:
Suppose three EC2 instances are registered.
Two are healthy and one is unhealthy.
The load balancer normally sends traffic only to the healthy targets.
Example diagram:
Target Group
├── EC2-1 → Healthy
├── EC2-2 → Healthy
└── EC2-3 → Unhealthy

Simple ga gurthupettuko:
Target Health = Can this target receive traffic?

---

54. What happens when all targets are unhealthy?
Answer:
If all registered targets fail their health checks, the load balancer has no healthy target available for normal routing.
The resulting client behavior depends on the load balancer and listener configuration.
This situation usually indicates an application, network, security group, or health-check configuration problem.
We should investigate target health, application ports, security groups, and health-check settings.
For example:
Suppose our ALB has three EC2 targets and all three fail health checks.
The ALB cannot normally forward requests to healthy application targets.
We should check the application and health-check configuration immediately.
Example diagram:
ALB
↓
Target Group
├── EC2-1 → Unhealthy
├── EC2-2 → Unhealthy
└── EC2-3 → Unhealthy
↓
No Healthy Target

Simple ga gurthupettuko:
All Unhealthy = No healthy backend available

---

55. What is the Tags section for a Load Balancer?
Answer:
Tags are key-value labels attached to AWS resources.
They help organize resources, identify environments, and support cost allocation and automation.
For example, we can tag a load balancer with Environment=Production.
Tags are useful when managing many AWS resources.
For example:
Suppose our company has development and production load balancers.
We can tag them with Environment=Dev and Environment=Prod.
This makes resource management easier.
Example diagram:
Load Balancer
↓
Tags
├── Environment = Production
├── Project = Shopping
└── Owner = DevOps

Simple ga gurthupettuko:
Tags = Identify and organize resources

---

56. What is Connection Draining?
Answer:
Connection draining refers to allowing existing connections or requests to finish before a target is removed from service.
In modern AWS terminology, this behavior is generally configured through deregistration delay.
It prevents new traffic from being sent to a deregistering target while existing work is allowed to complete.
This helps avoid abrupt interruption of user requests.
For example:

Suppose an EC2 instance needs maintenance.
We deregister it from the target group, and the load balancer stops sending new requests to it.
Existing requests get time to finish before the instance is removed.
Example diagram:
Target
↓
Deregister
↓
Stop New Traffic
↓
Finish Existing Traffic
↓
Removed
Simple ga gurthupettuko:
Connection Draining = Gracefully remove target

---

57. What is Weighted Target Group Forwarding?
Answer:
A listener can forward traffic to multiple target groups using configured weights when the feature is supported.
The weights determine the relative distribution of traffic between those target groups.
This is useful for controlled traffic shifting and application deployments.
It can be used to gradually move traffic from one version to another.
For example:
Suppose the old application is in one target group and the new application is in another.
We can initially send most traffic to the old group and a smaller amount to the new group.
Later, we can increase the new group's weight as the deployment becomes stable.
Example diagram:
ALB
↓
Listener
↓
Forward
├— Old TG → 90%
└— New TG → 10%
Simple ga gurthupettuko:
Weighted TG = Control traffic percentage between target groups

---

58. What is the relationship between Load Balancer, Listener, Target Group, and Target?
Answer:
The Load Balancer is the main entry point for client traffic.
A Listener accepts traffic on a configured protocol and port and evaluates routing rules.
The rule forwards matching requests to a Target Group.
The Target Group contains the backend targets that actually serve the application.
For example:
Suppose users access our website through an ALB on HTTPS port 443.
The HTTPS listener receives the request and forwards it to the web target group.
The target group then sends the request to a healthy EC2 instance.
Example diagram:
User
↓
Load Balancer
↓
Listener
↓
Listener Rule
↓
Target Group
↓
Healthy Target
↓
EC2
Simple ga gurthupettuko:
LB → Listener → Rule → Target Group → Target

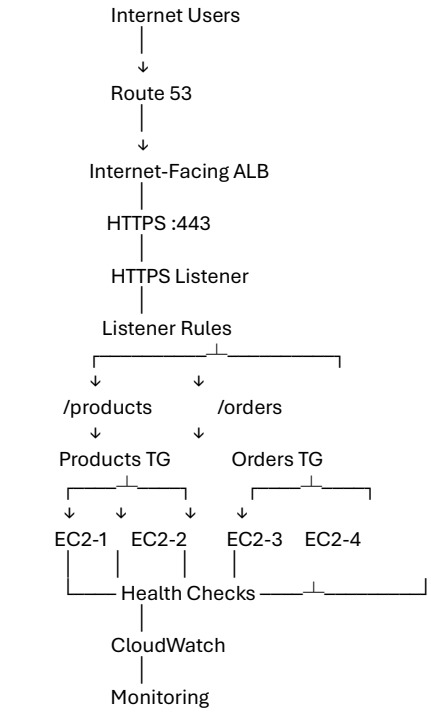
---

59. How does an ALB work with EC2?
Answer:
The ALB receives requests from users and sends them to registered EC2 targets.
The EC2 instances are placed inside a target group, and the ALB uses listeners and rules to decide where requests should go.
Health checks make sure traffic is normally sent to healthy targets.
This separates public traffic handling from the application servers.
For example:

Suppose our application has three EC2 instances running the same website.
We place them in a target group and connect that target group to the ALB listener.
Users access the ALB, and the ALB distributes requests among healthy EC2 instances.
Example diagram:
Users
↓
ALB
↓
Listener
↓
Web Target Group
├— EC2-1
├— EC2-2
└— EC2-3
Simple ga gurthupettuko:
ALB + EC2 = Public entry point + Backend servers

---

60. Explain Elastic Load Balancing completely with a real-world example.
Answer:
Suppose we have an online shopping application running on multiple EC2 instances.
We create an internet-facing ALB in multiple Availability Zones and configure an HTTPS listener on port 443.
The listener forwards requests to a target group containing healthy EC2 instances, while health checks remove unhealthy targets from normal traffic.
We can also use host/path rules, CloudWatch monitoring, security groups, access logging, and Route 53 for the complete architecture.
For example:
Suppose users open www.myshop.com and Route 53 points the domain to our ALB.
The ALB receives HTTPS requests and sends /products to the product service and /orders to the order service.
If one EC2 instance becomes unhealthy, the ALB stops sending normal traffic to that instance and uses the remaining healthy targets.
Example diagram:



Simple ga gurthupettuko:
ELB = Receive Traffic + Listen + Route + Health Check + Send to Healthy Targets